

LAB 4: VIBE CODING - LLM CHAT HISTORY & TRAJECTORY

Student SRN: PES1UG24CS384

Scenario: 15 (Connect Four vs AI)

Claude Chat Share Link: <https://claude.ai/share/56914cc2-0791-40fb-b8c1-a1e3acbea036>

Deliverable: (c) Complete Chat History Document / PDF

1. OVERVIEW & INITIAL BUG REPRODUCTION

Original Defect Identified:

In board.py, the winner(token) method checked only horizontal (0, 1) and vertical (1, 0) directions. Both diagonal directions (1, 1) down-right and (1, -1) down-left were omitted. Additionally, the AI in ai.py made purely random choices without detecting 1-move threats. The 'before.mp4' video was recorded demonstrating these original flaws.

2. ITERATION 1: TASK 1 - COMPLETE WIN DETECTION

User Prompt 1:

'In board.py, update the winner method to detect 4-in-a-row in all four directions: horizontal, vertical, diagonal down-right (1, 1), and diagonal down-left (1, -1). Ensure shorter sequences do not count as wins.'

LLM Implementation Applied in board.py:

```
def winner(self, token):
    directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token: continue
            for dr, dc in directions:
                if all(0 <= r + i*dr < ROWS and 0 <= c + i*dc < COLS and
                    self.grid[r + i*dr][c + i*dc] == token for i in range(4)):
                    return True
    return False
```

Result: Task 1 Verified & Committed (Commit: Task 1: Complete win detection).

3. ITERATION 2: TASK 2 - GAME TERMINATION & INPUT VALIDATION

User Prompt 2:

'In game.py, ensure proper input validation for column numbers (1 to 7), clean error messages when a column is full or invalid without crashing, and ensure winning moves and draws terminate the game cleanly.'

LLM Implementation Applied in game.py:

- Added bounds checking (1 to 7) and ValueError handling for non-numeric input.
- Added clean error message when a column is already full without crashing.
- Handled 'q' and KeyboardInterrupt cleanly to exit without unhandled exceptions.
- Ensured winning moves and full board conditions terminate the game loop immediately.

Result: Task 2 Verified & Committed (Commit: Task 2: Complete game termination).

4. ITERATION 3: TASK 3 - INTELLIGENT AI DECISION MAKING

User Prompt 3:

'In ai.py, upgrade the AI decision-making: first check if the AI has an immediate winning move and take it; second check if the opponent has an immediate winning move on their next turn and block it; otherwise choose safely from the remaining legal columns. Handle full columns safely.'

LLM Implementation Applied in ai.py:

- Step 1: Simulated AI move on board copy -> takes immediate winning drop.
- Step 2: Simulated Player move on board copy -> blocks immediate 4-in-a-row threat.
- Step 3: Filters unsafe moves (moves that hand opponent a win directly above).
- Step 4: Prefers center column (col index 3) when multiple safe choices exist.

Result: Task 3 Verified & Committed (Commit: Task 3: Improve AI decision making).

5. ITERATION 4: TASK 4 - MOVE-LEVEL FEEDBACK**User Prompt 4:**

'In game.py, add concise move-level feedback that prints which column the player or AI successfully placed their disc into, appearing exactly once per actual move.'

LLM Implementation Applied in game.py:

```
who = 'You' if self.turn == 'X' else 'AI'  
print(f'{who} placed a disc in column {col + 1}.')
```

Result: Task 4 Verified & Committed (Commit: Task 4: Add concise move-level feedback).

6. SUMMARY OF DELIVERABLES & VERIFICATION

- Tasks 1-4 completed in under 4 prompt iterations.
- 'before.mp4' (Original buggy/unblocked gameplay recorded).
- 'after.mp4' (Updated gameplay showing feedback, smart blocking, and diagonal win).
- All files pushed to personal repo: <https://github.com/rohilgirish/SE-Labs-PES1UG24CS384>